NUMBER SYSTEM (संख्या प्रणाली) **2**

संख्या प्रणालियों के प्रकार 2

दशमलव संख्या प्रणालियाँ *3*

बाइनरी संख्या प्रणालियाँ *3*

अष्टाधारी संख्या प्रणालियाँ *3*

हेक्साडेसिमल संख्या प्रणालियाँ *4*

संख्या प्रणालियों के बीच रूपांतरण *5*

एन्कोडिंग योजना 8

BOOLEAN LOGICS **9**

बूलियन तर्कशास्त्र और बीजगणित 9

बुनियादी तर्क द्वार 10

NOT गेट (इन्वर्टर) *10*

OR गेट *11*

AND गेट *11*

यूनिवर्सल (UNIVERSAL) गेट्स : *12*

NOR गेट: *12*

NAND गेट *12*

विशेष द्वार: 12

XOR गेट (एक्सक्लूसिव OR गेट) *12*

XNOR गेट (एक्सक्लूसिव NOR गेट) *13*

PROBLEM SOLVING **14**

समस्या समाधान का परिचय 14

एल्गोरिदम का प्रतिनिधित्व 16

FLOW CHART - एल्गोरिदम का दृश्य प्रतिनिधित्व 16

PSEUDO CODE 18

नियंत्रण का प्रवाह (FLOW OF CONTROL): **20**

अनुक्रम (SEQUENCE) 20

चयन (SELECTION) 20

दुहराव (REPETITION) 21

कोडन (CODING) 22

SYNTAX (वाक्यविन्यास) नियमों या व्याकरण का वह समूह है जो भाषा में कथनों के निर्माण को नियंत्रित करता है, जैसे वर्तनी, शब्दों का क्रम, विराम चिह्न आदि। 22

DECOMPOSITION (अपघटन) 22

NUMBER SYSTEM (संख्या प्रणाली)

इस पोस्ट में निम्नलिखित विषय शामिल हैं:

- **संख्या प्रणाली:**
  - **बाइनरी, ऑक्टल, दशमलव और हेक्साडेसिमल संख्या प्रणाली**
  - **संख्या प्रणालियों के बीच रूपांतरण.**
- **एनकोडिंग योजनाएँ:**
  - **ASCII, ISCII और यूनिकोड (UTF8, UTF32)**

संख्या प्रणालियाँ

संख्या प्रणाली संख्याओं को दर्शाने (लिखने) की एक विधि है। प्रत्येक संख्या प्रणाली में विशिष्ट वर्णों या अक्षरांकों का एक समूह होता है। इन अक्षरांकों की संख्या को संख्या प्रणाली का मूलांक या आधार कहते हैं।

संख्या प्रणालियों को स्थितिगत संख्या प्रणाली भी कहा जाता है क्योंकि प्रत्येक प्रतीक, अर्थात् अंक या वर्णमाला का मान, संख्या के भीतर उसकी स्थिति पर निर्भर करता है।

संख्या में हमारे द्वारा उपयोग किए जाने वाले दशमलव संख्याओं के समान एक भिन्नात्मक भाग भी हो सकता है।

किसी दी गई संख्या में पूर्णांक भाग में सबसे दाहिनी ओर स्थित प्रतीक का स्थान 0 होता है। पूर्णांक भाग में स्थान का मान (जिसे स्थिति मान भी कहा जाता है) दाईं से बाईं ओर 1 से बढ़ता है।

दूसरी ओर, संख्या के भिन्न भाग में पहले प्रतीक की स्थिति संख्या -1 है, जो भिन्न भाग को बाएं से दाएं पढ़ने पर 1 से घट जाती है।

उदाहरण के लिए

5	**6**	**8**	**. 9**	**2**
2	1	0	. -1	-2

5×10^{2} 6×10^{1} 8×10^{0} . 9×10^{-1} 2×10^{-2}

दशमलव प्रणाली में संख्या का प्रतिनिधित्व

संख्या प्रणालियों के प्रकार

कंप्यूटर प्रणाली में चार अलग-अलग प्रकार की संख्या प्रणालियों का उपयोग किया जाता है। ये हैं -

1. दशमलव संख्या प्रणालियाँ
2. बाइनरी संख्या प्रणालियाँ

3. अष्टाधारी संख्या प्रणालियाँ
4. हेक्साडेसिमल संख्या प्रणालियाँ

दशमलव संख्या प्रणालियाँ

दशमलव संख्या प्रणाली सबसे आम संख्या प्रणाली है। इसे आधार-10 संख्या प्रणाली के रूप में जाना जाता है। इसमें 10 अंक यानी विशिष्ट चिह्न (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) होते हैं।

उदाहरण के लिए : संख्या 568.92 को इस प्रकार दर्शाया गया है

5	**6**	**8**	**.**	**9**	**2**
2	1	0	**.**	-1	-2
5×10^{2}	6×10^{1}	8×10^{0}	**.**	9×10^{-1}	2×10^{-2}
500	**60**	**8**	.	**0.9**	**0.02**

दशमलव संख्या प्रणालियाँ

बाइनरी संख्या प्रणालियाँ

वह संख्या प्रणाली जिसमें केवल दो विशिष्ट चिह्न (द्वि) 0 और 1 हों, जिनका आधार 2 हो, द्विआधारी संख्या प्रणाली कहलाती है। उदाहरण के लिए 111001, 10011.111, 111001.1101।

दशमलव संख्याएं	**बाइनरी प्रतिनिधित्व (4-बिट)**
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

दशमलव संख्या अंकों का बाइनरी प्रतिनिधित्व (0 से 9)

अष्टाधारी संख्या प्रणालियाँ

अष्टाधारी संख्या प्रणाली को आधार-8 प्रणाली कहा जाता है क्योंकि इसमें कुल आठ अंक (0-7) होते हैं, तथा स्थितिगत मान
8 की घातों में व्यक्त किया जाता है। किसी भी अष्टाधारी संख्या को द्विआधारी संख्या में दर्शाने के लिए तीन द्विआधारी अंक अर्थात् $8 = 2^{3}$ पर्याप्त हैं।

उदाहरण : $(536)_{8}$, $(105)_{8}$

$(943)_{8}$ एक वैध अष्टाधारी संख्या क्यों नहीं है?

उत्तर : 943 में, 9 एक मान्य अष्टाधारी अंक नहीं है। केवल 0,1,2,3,4,5,6 और 7 ही किसी भी अष्टाधारी संख्या को दर्शाने के लिए मान्य अष्टाधारी अंक हैं।

अष्टाधारी संख्या प्रणाली क्यों?

जब आप एक बड़ी दशमलव संख्या को दर्शाते हैं, तो बाइनरी संख्याएँ इतनी बड़ी हो जाती हैं कि उन्हें प्रबंधित करना मुश्किल हो जाता है। इसलिए ऑक्टल संख्या प्रणाली विकसित की गई। ऑक्टल संख्या प्रणाली बाइनरी संख्याओं का एक संक्षिप्त निरूपण है।

अष्टाधारी अंक	बाइनरी प्रतिनिधित्व (3-बिट)
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

अष्टाधारी अंकों का बाइनरी प्रतिनिधित्व (0 से 7)

हेक्साडेसिमल संख्या प्रणालियाँ

हेक्साडेसिमल संख्या प्रणाली में 16 विशिष्ट चिह्न (0 - 9, A - F) होते हैं, और इसे आधार-16 प्रणाली कहा जाता है। हेक्साडेसिमल संख्या प्रणाली में, ध्यान दें कि दशमलव संख्याएँ **10 से 15 तक, A से F तक के** अक्षरों द्वारा दर्शाई जाती हैं ।

हेक्साडेसिमल संख्याओं का उपयोग बाइनरी संख्याओं के संक्षिप्त प्रतिनिधित्व के लिए भी किया जाता है।

हेक्साडेसिमल प्रणाली में, प्रत्येक अल्फ़ान्यूमेरिक अंक को 4 बाइनरी अंकों के समूह के रूप में दर्शाया जाता है क्योंकि 4 बिट्स ($16 = 2^4$) 16 अल्फ़ान्यूमेरिक प्रतीकों का प्रतिनिधित्व करने के लिए पर्याप्त हैं।

हेक्साडेसिमल संख्याओं के उदाहरण हैं $(23A.05)_{16}$, $(1C3)_{16}$, $(619B.A)_{16}$

दशमलव, षोडशआधारी, अष्टाधारी और बाइनरी का प्रतिनिधित्व

दशमलव संख्या	हेक्साडेसिमल प्रतीक	अष्टाधारी प्रतिनिधित्व	बाइनरी प्रतिनिधित्व (4-बिट)
0	0	0	0000
1	1	1	0001
2	2	2	0010
3	3	3	0011
4	4	4	0100
5	5	5	0101

6	6	6	0110
7	7	7	0111
8	8	10	1000
9	9	11	1001
10	ए	12	1010
11	बी	13	1011
12	सी	14	1100
13	डी	15	1101
14	ई	16	1110
15	एफ	17	1111

दशमलव -> षोडश आधारी -> अष्टाधारी -> बाइनरी प्रतिनिधित्व

संख्या प्रणालियों के बीच रूपांतरण

एक संख्या को एक आधार से दूसरे आधार में परिवर्तित करना

दशमलव संख्या प्रणाली से अन्य संख्या प्रणाली में रूपांतरण

किसी दशमलव संख्या को अन्य संख्या प्रणाली में बदलने के लिए, बार-बार विभाजन विधि का उपयोग किया जाता है।

दशमलव से बाइनरी रूपांतरण:

किसी दशमलव संख्या को बाइनरी संख्या में बदलने के लिए, बार-बार विभाजन विधि का उपयोग किया जाता है।

आधार 2 वाली दशमलव संख्या को बार-बार भाग दें, सभी शेषफलों को उल्टे क्रम में लिखें।

उदाहरण:

(150) 10 को बाइनरी संख्या में बदलें

आधार संख्या शेष

2	37	1	एलएसबी
			न्यूनतम महत्वपूर्ण बिट)
2	18	0	^
2	9	1	|
2	4	0	|
2	2	0	|
2	1	1	एमएसबी (सबसे महत्वपूर्ण बिट)
	0		

$(150)_{10} = (100101)_2$

उत्तर : $(150)_{10} = (100101)_2$

दशमलव से अष्टाधारी रूपांतरण:

दशमलव संख्या को अष्टाधारी संख्या में बदलने के लिए, बार-बार विभाजन विधि का उपयोग किया जा सकता है।

दशमलव संख्या को आधार 8 से बार-बार भाग दें, सभी शेषफलों को उल्टे क्रम में लिखें।

उदाहरण:

$(150)_{10}$ को अष्टाधारी में बदलें

आधार	संख्या	शेष	
8	150	6	एलएसबी (न्यूनतम महत्वपूर्ण बिट)
8	18	2	^
8	2	1	एमएसबी (सबसे महत्वपूर्ण बिट)
	0		

$(150)_{10} = (126)_{8}$

उत्तर : $(150)_{10} = (226)_{8}$

दशमलव से हेक्साडेसिमल रूपांतरण

किसी दशमलव संख्या को षोडश आधारी संख्या में बदलने के लिए हम बार-बार विभाजन विधि का उपयोग करते हैं।

आधार 16 वाली दशमलव संख्या को बार-बार भाग दें, सभी शेषफलों को उल्टे क्रम में लिखें।

उदाहरण:

$(170)_{10}$ को हेक्साडेसिमल में बदलें

आधार	संख्या	शेष	
16	165	5	एलएसबी (न्यूनतम महत्वपूर्ण बिट)
			^
16	10	10 -> A	एमएसबी (सबसे महत्वपूर्ण बिट)
	0		

$(165)_{10} = (A5)_{16}$

उत्तर : $(165)_{10} = (A5)_{16}$

बाइनरी, ऑक्टल और हेक्साडेसिमल संख्याओं को दशमलव संख्या में परिवर्तित करना

बाइनरी को दशमलव संख्या में परिवर्तित करना

सबसे दाएँ बिट से शुरू करें। प्रत्येक बिट को उसके स्थानीय मान से गुणा करें और फिर योगफल निकालें।

उदाहरण :-

बाइनरी संख्या 11011 को उसके दशमलव समकक्ष में बदलें।

$1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 * 2^0$

$16 + 8 + 0 + 2 + 1 = (27)_{10}$

अष्टाधारी संख्या को दशमलव संख्या में बदलना:

सबसे दाएँ बिट से शुरू करें। प्रत्येक बिट को उसके स्थानीय मान से गुणा करें और फिर योगफल निकालें।

उदाहरण:

अष्टाधारी संख्या 341 को उसके दशमलव समतुल्य में बदलें।

$= 3 \times 8^2 + 4 \times 8^1 + 1 \times 8^0$

$= 192 + 32 + 1$

$= (225)_{10}$

हेक्साडेसिमल संख्या को दशमलव संख्या में बदलना:

सबसे दाएँ बिट से शुरू करें। प्रत्येक बिट को उसके स्थानीय मान से गुणा करें और फिर योगफल निकालें।

उदाहरण

हेक्साडेसिमल संख्या $(21A)_{16}$ को उसके दशमलव समकक्ष में परिवर्तित करें।

21ए

$= 2 \times 16^2 + 1 \times 16^1 + A \times 16^0$

$= 2 \times 16^2 + 1 \times 16^1 + 10 * 16^0$

$= 512 + 16 + 10$

$= (538)_{10}$

बाइनरी संख्याओं के साथ अंकगणितीय ऑपरेशन:

बाइनरी जोड़

कंप्यूटर बाइनरी संख्याओं 0 और 1 के साथ काम करते हैं। वे केवल बाइनरी संख्याओं को ही समझते हैं। बाइनरी जोड़ के 5 मूल मामले हैं। ये हैं:-

1. 0+0 = 0
2. 0+1 = 1
3. 1+0 = 1
4. 1+1 = 10

यानी, बाइनरी 1+ बाइनरी 1 बराबर 0 और कैरी 1

5. 1+1+1 = 11

यानी, बाइनरी 1 + बाइनरी 1 बराबर 1 और कैरी 1

एन्कोडिंग योजना

वर्ण/स्ट्रिंग निरूपण:

कंप्यूटर संख्यात्मक और गैर-संख्यात्मक डेटा, जैसे अक्षर, विराम चिह्न और अन्य विशेष वर्ण, को संभाल सकता है। संख्यात्मक और गैर-संख्यात्मक वर्णों को दर्शाने के लिए कुछ पूर्वनिर्धारित कोड का उपयोग किया जाता है।

मानक एन्कोडिंग योजनाएँ इस प्रकार हैं:-

ASCII

ASCII का अर्थ है अमेरिकन स्टैंडर्ड कोड फॉर इन्फॉर्मेशन इंटरचेंज। ASCII-7 एक 7-बिट कोड है और 128 वर्णों का प्रतिनिधित्व कर सकता है। 7 बिट्स में से 3 ज़ोन बिट्स और 4 संख्यात्मक बिट्स हैं।

ASCII-8 256 वर्णों का प्रतिनिधित्व कर सकता है। यह ASCII-7 का विस्तारित रूप है।

ISCII: (सूचना विनिमय के लिए भारतीय मानक कोड)

कंप्यूटर पर भारतीय भाषाओं के उपयोग को सुविधाजनक बनाने के लिए कई प्रयास किए गए हैं । 1991 में, भारतीय मानक ब्यूरो ने ISCII को अपनाया। यह एक 8 बिट कोड है जो अंग्रेजी और भारतीय लिपियों के वर्णमाला को एक साथ उपयोग करने की अनुमति देता है। ISCII में कोड किए गए अक्षरों को प्रत्येक अक्षर के लिए 8 बिट्स की आवश्यकता होती है।

यूनिकोड (UNICODE)

यूनिकोड एक नया सार्वभौमिक कोडिंग मानक है जिसे सभी नए प्लेटफार्मों द्वारा अपनाया गया है। यह यूनिकोड कंसोर्टियम द्वारा प्रचारित किया जाता है जो एक गैर-लाभकारी संगठन है। यूनिकोड प्लेटफॉर्म, प्रोग्राम और भाषा के बावजूद हर अक्षर के लिए एक अद्वितीय संख्या प्रदान करता है। यह एक अक्षर कोडिंग प्रणाली है जिसे दुनिया भर में विविध भाषाओं के लिखित पाठों के आदान-प्रदान, प्रसंस्करण और प्रदर्शन का समर्थन करने के लिए डिज़ाइन किया गया है।

Boolean Logics

बूलियन तर्कशास्त्र और बीजगणित

बूलियन तर्क: NOT, AND, OR, NAND, NOR, XOR, सत्य सारणी, डी मॉर्गन के नियम और तर्क सर्किट

तार्किक कथन -
वह कथन या वाक्य जिसके केवल दो संभावित उत्तर हों - हाँ या नहीं, तार्किक कथन या सत्य फलन कहलाता है।

तार्किक चर / बूलियन चर :

वे चर जो तार्किक मान (स्थिरांक) सत्य या असत्य; 1 या 0; हाँ या नहीं रखते हैं, तार्किक चर कहलाते हैं।

बूलियन बीजगणित

दो मानों (सत्य या असत्य, हाँ या ना, 1 या 0) के बीजगणित को बूलियन बीजगणित कहा जाता है। यह डिजिटल सर्किट के विश्लेषण, डिज़ाइन और कार्यान्वयन में एक महत्वपूर्ण उपकरण है।

सत्य सारणी
सत्य सारणी एक सारणी है जो तार्किक चरों/कथनों के सभी संभावित मानों के साथ-साथ दिए गए मानों के संयोजन के सभी संभावित परिणामों को दर्शाती है। **तार्किक ऑपरेटर:** वे ऑपरेटर जिनका उपयोग तार्किक मानों (1 या 0; सत्य या असत्य) को संसाधित करने के लिए किया जाता है; तार्किक ऑपरेटर कहलाते हैं।
NOT, OR और AND तार्किक ऑपरेटर हैं।

ऑपरेटर नहीं	ओआर ऑपरेटर	AND ऑपरेटर
यह ऑपरेटर एकल चर पर कार्य करता है और इस ऑपरेशन को पूरकता कहा जाता है। यह ऑपरेशन चर के मान को उलट देता है ।	इस क्रिया को तार्किक योग कहते हैं और इसके लिए प्रयुक्त चिह्न + है। X+Y को X या Y पढ़ा जाता है। संभावित इनपुट और आउटपुट संयोजन नीचे दिए गए हैं:- 0 + 0 = 0 0 + 1 = 1 1 + 0 = 1 1 + 1 = 1	*AND संक्रिया को तार्किक गुणन कहते हैं। AND को दर्शाने के लिए* . (बिंदु) चिह्न का प्रयोग किया जाता है । *X* . Y को X AND Y पढ़ा जाता है। संभावित इनपुट और आउटपुट संयोजन नीचे दिए गए हैं:- 0 . 0 = 0 0 . 1 = 0 1 . 0 = 0 1 . 1 = 1

NOT, OR और AND तार्किक ऑपरेटरों के लिए सत्य तालिका:

NOT ऑपरेटर	OR ऑपरेटर	AND ऑपरेटर
x x' 1 0 0 1	xy x+y 0 0 0 0 1 1 1 0 1 1 1 1	xy xy 0 0 0 0 1 0 1 0 0 1 1 1

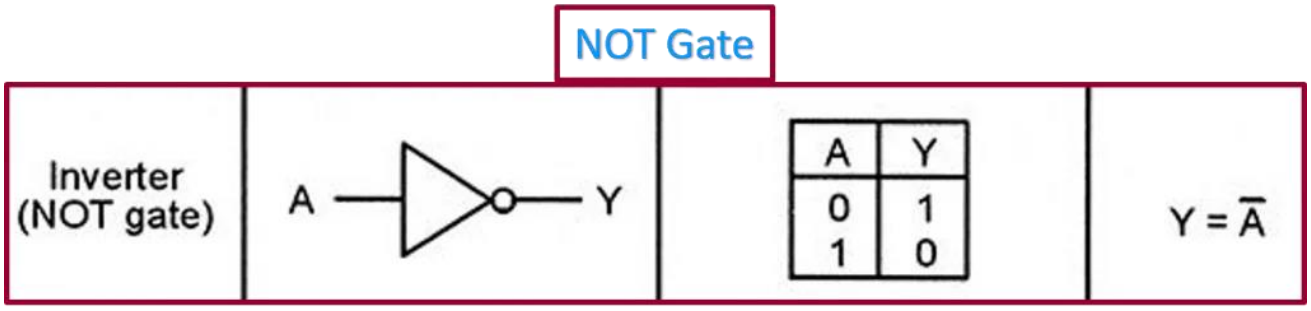
बुनियादी तर्क द्वार

गेट एक बुनियादी इलेक्ट्रॉनिक परिपथ है जो एक या एक से अधिक इनपुट सिग्नल पर काम करके आउटपुट सिग्नल उत्पन्न करता है।
लॉजिक गेट आठ प्रकार के होते हैं। इन्हें तीन श्रेणियों में वर्गीकृत किया गया है। ये हैं-

- बुनियादी द्वार
  - NOT GATE
  - OR गेट
  - AND गेट
- यूनिवर्सल गेट्स
  - NOR गेट
  - NAND गेट
- विशेष द्वार
  - XOR गेट
  - XNOR गेट

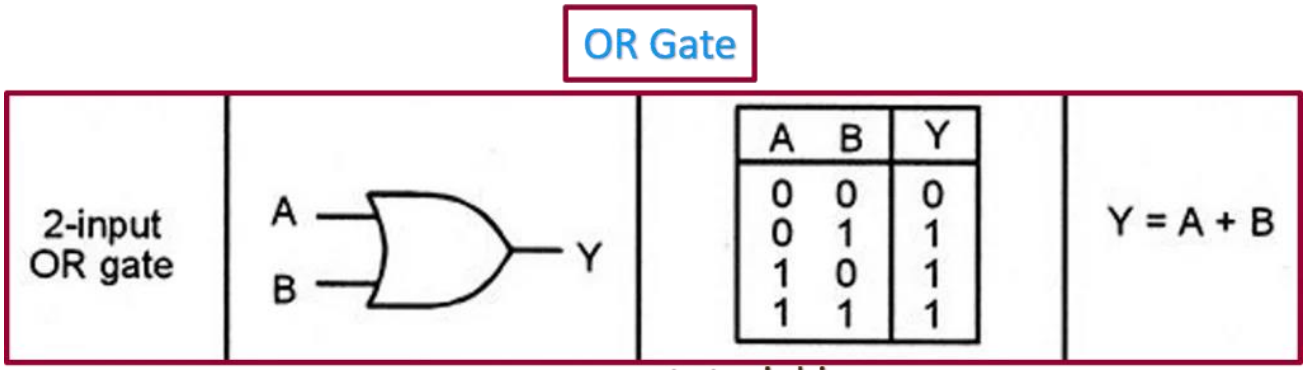
NOT गेट (इन्वर्टर)

इन्वर्टर एक गेट होता है जिसमें केवल एक इनपुट सिग्नल और एक आउटपुट सिग्नल होता है। आउटपुट अवस्था हमेशा इनपुट अवस्था के विपरीत होती है, अर्थात यदि इनपुट अवस्था 1 है तो आउटपुट अवस्था 0 होती है, और यदि इनपुट अवस्था 0 है तो आउटपुट अवस्था 1 होती है।



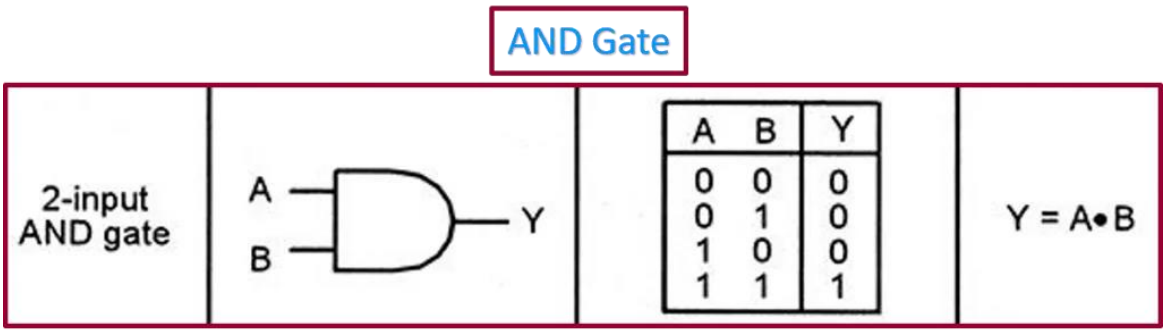
OR गेट

OR गेट में दो या अधिक इनपुट सिग्नल होते हैं, लेकिन केवल एक आउटपुट सिग्नल होता है। यदि सभी इनपुट 0 हैं, तो आउटपुट 0 होगा। यदि एक या अधिक इनपुट 1 हैं, तो आउटपुट 1 होगा।



AND गेट

AND गेट में दो या अधिक इनपुट सिग्नल हो सकते हैं, लेकिन एक आउटपुट सिग्नल। यदि कोई भी इनपुट 0 है, तो आउटपुट 0 होगा। यदि सभी इनपुट 1 हैं, तो आउटपुट 1 होगा।

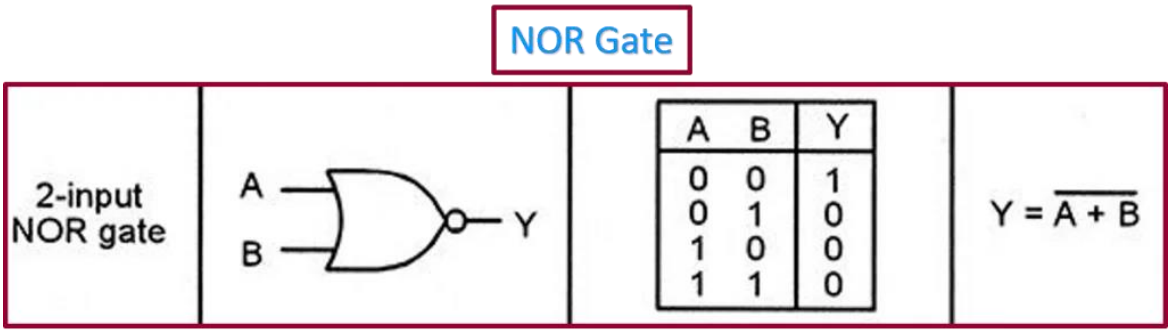


यूनिवर्सल (UNIVERSAL) गेट्स :

NOR और NAND को यूनिवर्सल गेट्स कहा जाता है। आप NOR और NAND गेट की मदद से बेसिक गेट्स (NOT, OR और AND) बना सकते हैं, इसीलिए इन्हें यूनिवर्सल गेट्स कहा जाता है।

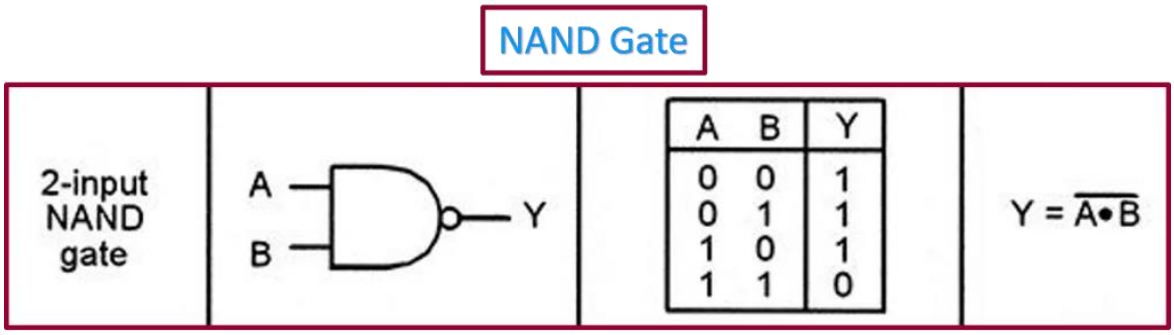
NOR गेट:

NOR गेट में दो या अधिक इनपुट सिग्नल होते हैं, लेकिन केवल एक आउटपुट सिग्नल होता है। NOR गेट एक उल्टा OR गेट है। यदि सभी इनपुट 0 हैं, तो आउटपुट 1 होता है। यदि कोई भी इनपुट 1 है, तो आउटपुट 0 होता है।



NAND गेट

NAND गेट में दो या अधिक इनपुट सिग्नल होते हैं, लेकिन केवल एक आउटपुट होता है। NAND गेट एक उल्टा AND गेट है। यदि सभी इनपुट 1 हैं, तो आउटपुट 0 होगा, अन्यथा यह आउटपुट 1 उत्पन्न करता है।



विशेष द्वार:

XOR गेट (एक्सक्लूसिव OR गेट)

XOR गेट में दो या अधिक इनपुट हो सकते हैं, लेकिन केवल एक आउटपुट। जब इनपुट में 1 की संख्या विषम हो, तो यह आउटपुट 1 उत्पन्न करता है, अन्यथा आउटपुट 0 होता है।

XOR Gate

2-input XOR gate		A B Y 0 0 0 0 1 1 1 0 1 1 1 0	$Y = A \oplus B$

XNOR गेट (एक्सक्लूसिव NOR गेट)

XNOR गेट में दो या अधिक इनपुट हो सकते हैं, लेकिन आउटपुट केवल एक ही होता है। जब इनपुट में 1 की संख्या सम होती है, तो यह आउटपुट 1 उत्पन्न करता है, अन्यथा आउटपुट 0 होता है।

XNOR Gate

2-input XNOR gate		A B Y 0 0 1 0 1 0 1 0 0 1 1 1	$Y = \overline{A \oplus B}$

Problem Solving

समस्या समाधान का परिचय

विषय:

- समस्या समाधान का परिचय:
  - समस्या समाधान के लिए चरण (समस्या का विश्लेषण, एल्गोरिदम विकसित करना, कोडिंग, परीक्षण और डिबगिंग)।
- एल्गोरिदम का प्रतिनिधित्व
  - प्रवाह चार्ट और
  - PSEUDO CODE, pseudo code
- सड़न

परिचय

कंप्यूटर एक ऐसी मशीन है जिसका उपयोग न केवल सॉफ्टवेयर विकसित करने के लिए किया जाता है , बल्कि इसका उपयोग विभिन्न दैनिक समस्याओं को हल करने के लिए भी किया जाता है।

कंप्यूटर अकेले किसी समस्या का समाधान नहीं कर सकते। वे हमारे द्वारा दिए गए चरण-दर-चरण निर्देशों के आधार पर समस्या का समाधान करते हैं।

इस प्रकार, किसी समस्या को हल करने में कंप्यूटर की सफलता इस बात पर निर्भर करती है कि हम कितनी सही और सटीक तरीके से -

- समस्या की पहचान (परिभाषित) करना
- एक एल्गोरिथ्म का डिजाइन और विकास करना और
- एल्गोरिथ्म (समाधान) को कार्यान्वित करते समय किसी भी प्रोग्रामिंग भाषा का उपयोग करके प्रोग्राम विकसित करें।

इस प्रकार समस्या समाधान एक आवश्यक कौशल है जो कंप्यूटर विज्ञान के छात्र को आना चाहिए।

समस्या समाधान के लिए कदम-

1. समस्या का विश्लेषण

समस्याओं का विश्लेषण करने का अर्थ है कि समाधान ढूँढने से पहले समस्या को अच्छी तरह समझना। समस्या का विश्लेषण करने से यह समझने में मदद मिलती है कि हमारे प्रोग्राम को कौन से इनपुट स्वीकार करने चाहिए और कौन से आउटपुट देने चाहिए।

2. एक एल्गोरिथ्म विकसित करना

किसी समस्या के लिए प्रोग्राम कोड लिखने से पहले उसका समाधान तैयार करना ज़रूरी है। समाधान प्राकृतिक भाषा में प्रस्तुत किया जाता है और उसे एल्गोरिथम कहा जाता है।

एल्गोरिदम (ALGORITHM) : सटीक चरणों का एक समूह जिसका पालन करने पर समस्या का समाधान हो जाता है या अपेक्षित कार्य पूरा हो जाता है।

3. कोडिंग

कोडिंग, एल्गोरिदम को प्रोग्राम में परिवर्तित करने की प्रक्रिया है जिसे कंप्यूटर द्वारा वांछित समाधान उत्पन्न करने के लिए समझा जा सकता है।

आप प्रोग्राम लिखने के लिए किसी भी उच्च स्तरीय प्रोग्रामिंग भाषा का उपयोग कर सकते हैं।

4. परीक्षण और डिबगिंग (Debugging)

बनाए गए प्रोग्राम का विभिन्न मापदंडों पर परीक्षण किया जाना चाहिए।

- कार्यक्रम को उपयोगकर्ता की आवश्यकताओं को पूरा करना चाहिए।
- इसे अपेक्षित समय के भीतर जवाब देना होगा।
- इसे सभी संभावित इनपुट के लिए सही आउटपुट उत्पन्न करना चाहिए।
- वाक्यविन्यास संबंधी त्रुटियों की उपस्थिति में, कोई आउटपुट प्राप्त नहीं होगा।
- यदि उत्पन्न आउटपुट गलत है, तो प्रोग्राम में तार्किक त्रुटियों (यदि कोई हो) की जांच की जानी चाहिए।

सॉफ्टवेयर परीक्षण विधियाँ हैं

- इकाई या घटक परीक्षण (UNIT TESTING)
- एकीकरण परीक्षण (INTEGRATION TESTING)
- सिस्टम परीक्षण (SYSYTEM TESTING) और
- स्वीकृति परीक्षण (ACCEPTANCE TESTING)

डिबगिंग - परीक्षण चरणों में पाई गई त्रुटियों या दोषों को डिबग या सुधारा जाता है और प्रोग्राम का पुनः परीक्षण किया जाता है। यह तब तक जारी रहता है जब तक प्रोग्राम से सभी त्रुटियाँ दूर नहीं हो जातीं।

एल्गोरिथ्म (ALGORITHM)

एल्गोरिथ्म अनुक्रम का एक समूह है जिसका अनुसरण किसी समस्या को हल करने के लिए किया जाता है।

'साइकिल चलाना' गतिविधि के लिए एल्गोरिथ्म:

1) साइकिल को स्टैंड से हटा दें,
2) साइकिल की सीट पर बैठ जाएं,
3) पैडल मारना शुरू करें,
4) जब भी आवश्यकता हो ब्रेक का उपयोग करें और
5) गंतव्य पर पहुंचने पर रुक जाएं।

दो संख्याओं के GCD की गणना के लिए एल्गोरिथ्म:

चरण 1: वे संख्याएँ (विभाजक) ज्ञात कीजिए जो दी गई संख्याओं को विभाजित कर सकें।

चरण 2: फिर इन दो सूचियों में से सबसे बड़ी सामान्य संख्या ज्ञात कीजिए।

<u>वांछित परिणाम प्राप्त करने के लिए आवश्यक चरणों के एक सीमित क्रम को एल्गोरिथम कहते हैं। एल्गोरिथम का एक निश्चित आरंभ और एक निश्चित अंत होता है, और इसमें चरणों की एक सीमित संख्या होती है।</u>

एक अच्छे एल्गोरिथम की विशेषताएँ

- **परिशुद्धता** - चरणों को सटीक रूप से बताया या परिभाषित किया गया है।
- **विशिष्टता** - प्रत्येक चरण के परिणाम विशिष्ट रूप से परिभाषित होते हैं और केवल इनपुट और पूर्ववर्ती चरणों के परिणाम पर निर्भर करते हैं।
- **परिमितता** - एल्गोरिथ्म हमेशा चरणों की एक सीमित संख्या के बाद रुक जाता है।
- **इनपुट** - एल्गोरिथम कुछ इनपुट प्राप्त करता है।
- **आउटपुट** - एल्गोरिथ्म कुछ आउटपुट उत्पन्न करता है।

एल्गोरिथ्म लिखते समय निम्नलिखित को स्पष्ट रूप से पहचानना आवश्यक है:

- उपयोगकर्ता से लिया जाने वाला इनपुट.
- वांछित परिणाम प्राप्त करने के लिए किया जाने वाला प्रसंस्करण या गणना।
- उपयोगकर्ता द्वारा वांछित आउटपुट.

एल्गोरिदम का प्रतिनिधित्व

किसी एल्गोरिथ्म को दर्शाने के दो सामान्य तरीके हैं -

- प्रवाह चार्ट (**FLOW CHART)**
- PSEUDO CODE

<u>FLOW CHART - एल्गोरिदम का दृश्य प्रतिनिधित्व</u>

FLOW CHART किसी एल्गोरिथम का दृश्य निरूपण होता है। FLOW CHART बक्सों, हीरों और अन्य आकृतियों से बना एक आरेख होता है, जो तीरों द्वारा जुड़ा होता है। प्रत्येक आकृति समाधान प्रक्रिया के एक चरण का प्रतिनिधित्व करती है और तीर चरणों के बीच क्रम या कड़ी को दर्शाता है। FLOW CHART बनाने के लिए मानकीकृत प्रतीक होते हैं।

प्रारंभ/अंत - इसे "टर्मिनेटर" प्रतीक भी कहा जाता है। यह दर्शाता है कि प्रवाह कहाँ से शुरू और कहाँ समाप्त होता है।

प्रक्रिया - इसे "क्रिया प्रतीक" भी कहा जाता है, यह एक प्रक्रिया, क्रिया या एक चरण का प्रतिनिधित्व करता है।
निर्णय - एक निर्णय या शाखा बिंदु, आमतौर पर हाँ/ना या सही/गलत प्रश्न पूछा जाता है, और उत्तर के आधार पर, पथ दो शाखाओं में विभाजित हो जाता है।

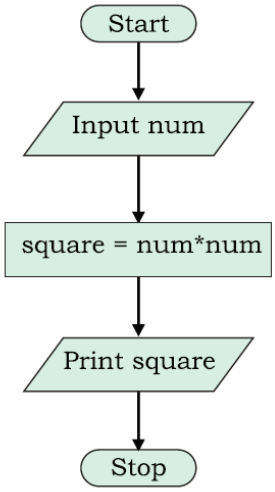
इनपुट / आउटपुट - इसे डेटा प्रतीक भी कहा जाता है, इस समांतर चतुर्भुज आकार का उपयोग डेटा को इनपुट या आउटपुट करने के लिए किया जाता है।

तीर - आकृतियों के बीच प्रवाह का क्रम दिखाने के लिए कनेक्टर।

Symbol	Name	Function
	Start/end	An oval represents a start or end point.
	Arrows	A line is a connector that shows relationships between the representative shapes.
	Input/Output	A parallelogram represents input or ouptut.
	Process	A rectangle represents a process.
	Decision	A diamond indicates a decision.

प्रश्न: किसी संख्या का वर्ग ज्ञात करने के लिए एक एल्गोरिथ्म लिखें।
किसी संख्या का वर्ग ज्ञात करने के लिए एल्गोरिथ्म।
चरण 1: एक संख्या दर्ज करें और उसे num में संग्रहीत करें।
चरण 2: num * num की गणना करें और उसे वर्ग में संग्रहीत करें।
चरण 3: वर्ग प्रिंट करें।

किसी संख्या का वर्ग ज्ञात करने के लिए एल्गोरिथ्म को फ्लोचार्ट का उपयोग करके चित्रात्मक रूप से दर्शाया जा सकता है



PSEUDO CODE

स्यूडोकोड (PSEUDO CODE) ((उच्चारण: सू-डो-कोहद) एल्गोरिथम को दर्शाने का एक और तरीका है। इसे एक अनौपचारिक भाषा माना जाता है जो प्रोग्रामर्स को एल्गोरिथम लिखने में मदद करती है। यह उन निर्देशों का विस्तृत विवरण है जिनका कंप्यूटर को एक निश्चित क्रम में पालन करना होता है।

- यह मानव द्वारा पढ़ने के लिए है और इसे सीधे कंप्यूटर द्वारा निष्पादित नहीं किया जा सकता।
- छद्मकोड लिखने के लिए कोई विशिष्ट मानक मौजूद नहीं है।
- शब्द "छद्म" का अर्थ है "वास्तविक नहीं", इसलिए "छद्मकोड" का अर्थ है "वास्तविक कोड नहीं"।

PSEUDO CODE में कीवर्ड का उपयोग किया जाता है:

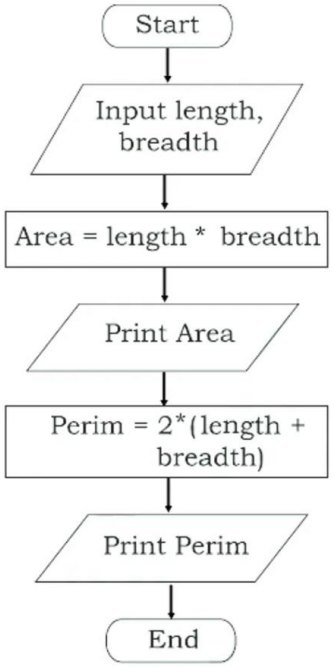
- इनपुट
- गणना
- प्रिंट
- वेतन वृद्धि
- घटती
- यदि/अन्यथा
- जबकि
- सही/गलत

प्रश्न: छद्मकोड और फ्लोचार्ट दोनों का उपयोग करके एक आयत के क्षेत्रफल और परिधि की गणना करने के लिए एक एल्गोरिदम लिखें।

एक आयत के क्षेत्रफल और परिधि की गणना के लिए PSEUDO CODE।

इनपुट लंबाई
इनपुट चौड़ाई
कंप्यूट क्षेत्रफल = लंबाई * चौड़ाई
प्रिंट क्षेत्रफल
कंप्यूट परिम = 2 * (लंबाई + चौड़ाई)
प्रिंट परिम

इस एल्गोरिथम के लिए फ्लोचार्ट



स्यूडोकोड के लाभ

- किसी प्रोग्राम का PSEUDO CODE, इच्छित प्रोग्राम की मूल कार्यक्षमता को दर्शाने में मदद करता है।
- कोड को पहले मानव-पठनीय भाषा में लिखकर, प्रोग्रामर किसी भी महत्वपूर्ण चरण को छोड़ने से बच जाता है।
- गैर-प्रोग्रामर्स के लिए, वास्तविक प्रोग्रामों को पढ़ना और समझना कठिन होता है, लेकिन PSEUDO CODE उन्हें चरणों की समीक्षा करने में मदद करता है, ताकि यह पुष्टि की जा सके कि प्रस्तावित कार्यान्वयन वांछित आउटपुट प्राप्त करने जा रहा है।

नियंत्रण का प्रवाह (FLOW OF CONTROL):

नियंत्रण प्रवाह, प्रवाह चार्ट में दर्शाए गए प्रक्रिया प्रवाह को दर्शाता है। प्रक्रिया निम्न प्रकार प्रवाहित हो सकती है:

- अनुक्रम (SEQUENCE)
- चयन (SELECTION)
- दुहराव (REPETITION)

अनुक्रम (SEQUENCE)

एल्गोरिदम के अनुक्रमिक चरणों (अर्थात कथनों) को एक के बाद एक निष्पादित किया जाता है।

चयन (SELECTION)

चयन में, एल्गोरिथ्म के चरण शर्तों पर निर्भर होते हैं, अर्थात किसी शर्त के परिणाम के आधार पर वैकल्पिक कथनों में से किसी एक का चयन किया जाता है।

सशर्तों का उपयोग संभावनाओं की जाँच के लिए किया जाता है। प्रोग्राम एक या एक से अधिक शर्तों की जाँच करता है और शर्त के सही या गलत मान के आधार पर संचालन (क्रियाओं का क्रम) करता है।

एल्गोरिथम में सशर्तों को इस प्रकार लिखा जाता है:

यदि सत्य है तो
शर्त के सत्य/पूरा होने पर उठाए जाने वाले कदम,
अन्यथा
शर्त के असत्य/पूरा न होने पर उठाए जाने वाले कदम

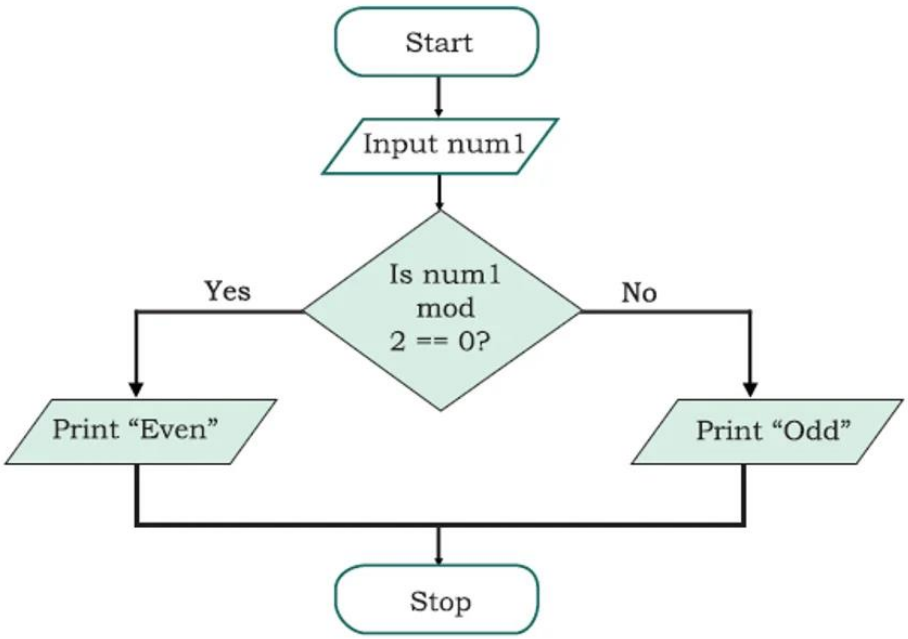
प्रश्न: यह जांचने के लिए एक एल्गोरिथ्म लिखें कि कोई संख्या विषम है या सम।

• इनपुट: कोई भी संख्या
• प्रक्रिया: जांचें कि संख्या सम है या नहीं
• आउटपुट: संदेश “सम” या “विषम”

एल्गोरिथ्म का PSEUDO CODE इस प्रकार लिखा जा सकता है:

PRINT “संख्या दर्ज करें”
INPUT संख्या
यदि संख्या MOD 2 == 0 तो
PRINT “संख्या सम है”
अन्यथा
PRINT “संख्या विषम है”

एल्गोरिथ्म का FLOW CHART प्रतिनिधित्व



दुहराव (REPETITION)

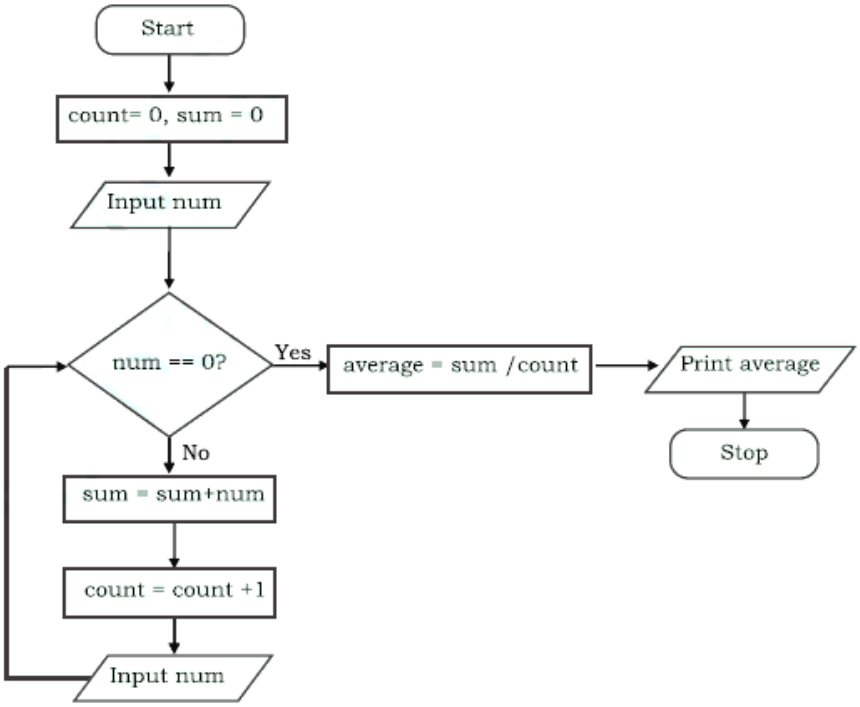
पुनरावृत्ति का प्रयोग तब किया जाता है, जब हम किसी कार्य को निश्चित संख्या में बार-बार करना चाहते हैं।

प्रश्न: PSEUDO CODE लिखें और एक FLOW CHART बनाएँ जो उपयोगकर्ता द्वारा शून्य दर्ज करने तक संख्याओं को स्वीकार करे और फिर उनका औसत ज्ञात करे।

PSEUDO CODE इस प्रकार है:

चरण 1: गणना = 0, योग = 0 सेट करें
चरण 2: संख्या इनपुट करें
चरण 3: जब तक संख्या 0 के बराबर न हो, चरण 4 से 6 दोहराएँ
चरण 4: योग = योग + संख्या
चरण 5: गणना = गणना + 1
चरण 6: संख्या इनपुट करें
चरण 7: औसत की गणना करें = योग/गणना
चरण 8: औसत प्रिंट करें

FLOW CHART प्रतिनिधित्व है



कोडन (CODING)

एक बार एल्गोरिथ्म तय हो जाने के बाद, उसे प्रोग्रामर द्वारा चुनी गई उच्च-स्तरीय प्रोग्रामिंग भाषा में कोडित किया जाना चाहिए। निर्देशों का क्रमबद्ध सेट उस प्रोग्रामिंग भाषा में उसके सिंटैक्स का पालन करते हुए लिखा जाता है।

SYNTAX (वाक्यविन्यास) नियमों या व्याकरण का वह समूह है जो भाषा में कथनों के निर्माण को नियंत्रित करता है, जैसे वर्तनी, शब्दों का क्रम, विराम चिह्न आदि।

स्रोत कोड (SOURCE CODE): उच्च स्तरीय भाषा में लिखे गए प्रोग्राम को स्रोत कोड कहा जाता है।

हमें स्रोत कोड को कम्पाइलर या इंटरप्रेटर का उपयोग करके मशीन भाषा में अनुवाद करना होगा ताकि इसे कंप्यूटर द्वारा समझा जा सके।

DECOMPOSITION (अपघटन)

अपघटन (DECOMPOSITION) एक जटिल समस्या को छोटी-छोटी उप-समस्याओं में 'विघटित' करने या तोड़ने की प्रक्रिया है। यह तब मददगार होता है जब हमें किसी बड़ी या जटिल समस्या का समाधान करना होता है।

- किसी जटिल समस्या को उप-समस्याओं में विभाजित करने का अर्थ यह भी है कि प्रत्येक उप-समस्या की विस्तार से जांच की जा सकती है।

- प्रत्येक उपसमस्या को स्वतंत्र रूप से और अलग-अलग व्यक्तियों (या टीमों) द्वारा हल किया जा सकता है।
- विभिन्न उप-समस्याओं पर अलग-अलग टीमों का काम करना भी लाभदायक हो सकता है, क्योंकि विशिष्ट उप-समस्याओं को उन टीमों को सौंपा जा सकता है जो ऐसी समस्याओं को सुलझाने में विशेषज्ञ हैं।

एक बार जब व्यक्तिगत उप-समस्याएं हल हो जाती हैं, तो उनकी सत्यता की जांच करना तथा पूर्ण समाधान प्राप्त करने के लिए उन्हें एकीकृत करना आवश्यक होता है